

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Computer Vision with OpenCV COURSE CODE: DSA02

COURSE OUTCOME COVERED

CO5: Apply computer vision techniques to real-world applications such as face recognition, tracking, medical imaging, and autonomous systems. (BL3)

Assessment Tool Weightage: 30%

QUESTIONS: 2 Total Marks:30 Annexure A

Pseudocode Writing Task (Algorithm Design)

Code of Conduct:

I Shri Harshini Devi A - 192524104 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation	
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach	
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs	
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear	

Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient	
--------------	-----	--	--	--------------------------------	------------------------------	--

Annexure A

Pseudocode Writing Task (Algorithm Design)

1. Scenario: Design an algorithm to detect faces in an image and recognize them using the Eigenfaces method.

Task:

- A. Input: A grayscale image (size $M \times N$)
- B. Output: Bounding boxes of detected faces and corresponding recognized names
- C. Requirements: Handle multiple faces in the same image and varying lighting conditions

Pseudocode Task:

1. Preprocess the image (grayscale normalization, histogram equalization)
2. Detect faces using a sliding window or Viola-Jones detector
3. Extract features using Eigenfaces
4. Compare with database of known faces using Euclidean distance
5. Output bounding boxes with recognized labels

2. Scenario: Design an algorithm to detect road lanes in a real-time driving video sequence.

Task:

- A. Input: Video frames from a front-facing camera
- B. Output: Lane lines highlighted on each frame
- C. Requirements: Robust to shadows, curves, and partial occlusion

Pseudocode Task:

1. Capture video frame
2. Convert frame to grayscale
3. Apply Gaussian smoothing to reduce noise
4. Perform edge detection (Canny)
5. Apply Hough Transform to detect straight/curved lines
6. Overlay detected lanes on the original frame
7. Repeat for all frames

Eigenfaces-Based Face Detection and Recognition

1. Objective

The aim is to detect multiple faces in a grayscale image and recognize each detected face by comparing its Eigenface features with a database of known faces.

2. Input and Output

- **Input:** Grayscale image of size $M \times N$
- **Output:** Bounding box (x, y, width, height) and recognized name for each face.

3. Algorithm

Step 1: Preprocessing

1. Convert the input image to grayscale.
2. Normalize pixel intensity values.
3. Apply **histogram equalization** to reduce the effect of varying lighting conditions.
4. Remove noise using a suitable filter if required.

Step 2: Face Detection

- Use a **Viola-Jones detector** or sliding-window technique.
- Scan the image and identify regions containing faces.
- Store the bounding box of every detected face.

Step 3: Face Alignment and Normalization

- Crop each detected face using its bounding box.
- Resize all face images to the same size.
- Normalize the intensity of each cropped face.

Step 4: Eigenfaces Feature Extraction

1. Convert each training face into a vector.
2. Calculate the **mean face**.
3. Subtract the mean face from every training image.
4. Calculate the covariance matrix.
5. Find the principal eigenvectors.
6. Select the eigenvectors with the largest eigenvalues as **Eigenfaces**.
7. Project each training face onto the Eigenface space to obtain its feature vector.

Step 5: Recognition

- Project each detected face onto the Eigenface space.
- Calculate the **Euclidean distance** between its feature vector and the feature vectors stored in the database.
- Select the face with the minimum distance.
- If the distance is below a predefined threshold, assign the corresponding person's name.
- Otherwise, label it as **Unknown**.

Step 6: Output

- Draw a bounding box around every detected face.
- Display the recognized name above each bounding box.

4. Pseudocode

Algorithm Eigenface_Face_Recognition(Image)

Input: Grayscale image I of size $M \times N$
Output: Bounding boxes and recognized names

1. $I \leftarrow \text{Normalize}(I)$
2. $I \leftarrow \text{HistogramEqualization}(I)$
3. $\text{Faces} \leftarrow \text{ViolaJonesDetector}(I)$
4. Load training face database
5. Calculate MeanFace
6. Calculate Eigenfaces from training images
7. For each face F in Faces do
 - a. Crop F using its bounding box
 - b. Resize F to standard size
 - c. Normalize F
 - d. Subtract MeanFace
 - e. Project F onto Eigenface space
 - f. Calculate Euclidean distance with all database faces
 - g. Find minimum distance
 - h. If minimum distance $<$ Threshold then
 Name $\leftarrow$ Corresponding database name
Else
 Name $\leftarrow$ "Unknown"
 - i. Draw bounding box around F
 - j. Display Name
- End For
8. Return bounding boxes and names

End Algorithm

5. Handling Multiple Faces

The **Viola-Jones detector** returns multiple bounding boxes. The Eigenface recognition process is then performed **independently for every detected face**, allowing several people to be recognized in the same image.

6. Handling Varying Lighting Conditions

Lighting variations can reduce Eigenface recognition accuracy. To improve robustness:

- Histogram equalization
 - Intensity normalization
 - Proper face alignment
 - Threshold tuning
- can be applied during preprocessing.

7. Flow

Input Image → Grayscale Normalization → Histogram Equalization → Face Detection → Face Cropping & Resizing → Eigenface Feature Extraction → Euclidean Distance → Recognition → Bounding Box + Name

8. Advantages

- Simple and computationally efficient.
- Works well with a controlled face database.
- Can recognize multiple faces.
- Eigenfaces reduce the dimensionality of face images.

9. Limitation

Eigenfaces are sensitive to **large changes in lighting, pose, facial expression, and occlusion**, so preprocessing and proper threshold selection are important.

2. Road Lane Detection in Real-Time Video

1. Objective

The aim is to detect road lane markings from a **front-facing camera video** and highlight the detected lanes on every frame, even under shadows, curves, and partial occlusion.

2. Input and Output

- **Input:** Video frames captured from a front-facing camera.
- **Output:** Video frames with detected **lane lines highlighted**.

3. Algorithm

Step 1: Capture Video Frame

- Read one frame at a time from the driving video.

Step 2: Preprocessing

1. Convert the frame to **grayscale**.
2. Apply **Gaussian smoothing** to remove noise.
3. Apply brightness/contrast normalization if required to handle shadows.

Step 3: Edge Detection

- Apply the **Canny edge detector** to identify strong edges corresponding to lane markings.

Step 4: Region of Interest (ROI)

- Select the road region, usually the lower portion of the frame.
- Ignore irrelevant regions such as the sky and buildings.

Step 5: Lane Detection

- Apply the **Hough Transform** to detect straight lane segments.
- For curved roads, use **probabilistic Hough transform**, polynomial fitting, or a sliding-window approach to estimate curved lanes.

Step 6: Handle Occlusion

- Track previously detected lane positions across consecutive frames.
- Use the previous frame's lane information when a lane is temporarily blocked.

Step 7: Overlay

- Draw the detected lane lines on the original video frame.

- Highlight the lane region if required.

Step 8: Repeat

- Continue processing until all video frames are completed.

4. Pseudocode

Algorithm Lane_Detection(Video)

Input: Video frames from front-facing camera

Output: Frames with highlighted lane lines

1. Open Video
2. While Video has frames do
 - a. Frame $\leftarrow$ CaptureFrame(Video)
 - b. Gray $\leftarrow$ ConvertToGrayscale(Frame)
 - c. Smooth $\leftarrow$ GaussianBlur(Gray)
 - d. Edges $\leftarrow$ CannyEdgeDetection(Smooth)
 - e. ROI $\leftarrow$ SelectRoadRegion(Edges)
 - f. Lines $\leftarrow$ HoughTransform(ROI)
 - g. If curved lanes are detected then
 - FitCurve(Lines)
 - End If
 - h. Use previous frame information to handle partial occlusion
 - i. Draw detected lane lines on Frame
 - j. Display Frame

End While

3. Close Video

End Algorithm

5. Handling Special Conditions

Problem	Technique
Shadows	Histogram/brightness normalization and adaptive thresholding
Curved roads	Polynomial curve fitting or sliding-window method

Problem	Technique
Partial occlusion	Lane tracking using previous frames
Noise	Gaussian smoothing
Irrelevant objects	Region of Interest (ROI)
Broken lane markings	Hough Transform combines multiple line segments

6. Flow

Video Frame → Grayscale → Gaussian Blur → Canny Edge Detection → ROI Selection → Hough Transform/Curve Fitting → Lane Tracking → Overlay Lanes → Display Frame → Next Frame

7. Advantages

- Can process video frames continuously.
- Canny detection provides clear road edges.
- ROI reduces unnecessary computation.
- Tracking improves stability during temporary occlusion.
- Curve fitting allows detection of non-straight lanes.